Musil

Scripting Language for Sound And Music Computing

User Manual version 0.7

Musil is a small scripting language for sound and music computing: Scheme underneath, Tcl on the surface. Numbers are vectors, functions are values, code is data. The language is one C++ header with no dependencies; the libraries add the operating system, linear algebra and machine learning, signal processing, real-time sound and plotting, and the IDE puts all of it in a window.

Contents

1 Overview

Musil is a language for people who make music with computation: composers, researchers, students. It is deliberately small. The core language fits in one C++ file of under a thousand lines, and every feature in this manual can be reached from that core by loading a library. The design has three commitments:

- **Numbers are vectors.** A scalar is a vector of size one; `(+ v 10)` adds ten to every element; `(sin v)` is elementwise. Signal processing and numerical work are written as arithmetic, and run at C++ speed.
- **Functions are values.** They are created anywhere, stored in lists, passed to `map`, applied to too few arguments to make partial applications, and called in tail position without growing the stack.
- **Code is data.** A program is a list; `'(+ 1 2)` is a list you can build, inspect and `eval`.

The surface is what makes it comfortable: each line is a command, so `print "x ="x` needs no parentheses around it; `( )` is a list; `{ }` is a block of lines; `(expr (a + b * 2))` gives you infix arithmetic when a formula reads better that way.

1.1 Two ways to run Musil

The command line. `musil file.mu` runs a file; `musil` alone is a REPL; `musil a.mu b.mu - x y` runs several files in one interpreter with `args` bound to `(x y)`; `musil -e "print 42"` evaluates a string.

The IDE. *Musil* the application is an editor with syntax colouring where Cmd-Enter runs the block under the cursor, a console with an input line, the list of your variables, a documentation search, and plots and controls in windows of their own. It runs the same interpreter. See section 3.

1.2 Building and installing

```
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build
ctest --test-dir build          # every library has a test and a
    reference
cmake --install build           # musil -> /usr/local/bin,  *.mu and
    help.txt -> ~/.musil
./deploy_macos.sh               # macOS: dist/Musil.app
```

The build fetches FLTK 1.4, the one external dependency, used by the plot library, the controls window and the IDE; `-DMUSIL_FLTK=OFF` builds everything else with no dependency at all. Libraries are found by `load` in `~/.musil` after installing, in any directory listed in the `MUSIL_PATH` environment variable, or next to the file that loads them; during development `MUSIL_PATH=src` points at the source tree.

1.3 Getting help

`(help name)` prints the signature and description of any builtin or library function, taken from the same comments this manual is generated from:

```
> help range
  (range n)  (range a b)  (range a b step)
  a vector of evenly spaced numbers, end excluded
```

The files `examples/reference_*.mu` are runnable tours of the core and of each library; `examples/reference.mu` is the best place to start.

2 The language

2.1 Reading a program

A program is a sequence of lines. Each line is a command: its first word is a function and the rest are its arguments.

```
print "hello"                    # (print "hello")
var x 10                        # (var x 10)
print "x + 1 =" (+ x 1)         # a parenthesised form is a
    sub-expression
```

Inside parentheses, newlines are ordinary whitespace, so a long call can span lines. Braces group lines into a block, which is what `if`, `while` and function bodies take:

```
if (> x 5) {
    print "big"
    var y (* x 2)
}
```

A line holding a single parenthesised form is that form; a line holding a single literal is that value; a line holding a single *symbol* calls it (`ctr` calls the function `ctr`, as in Tcl). A comment runs from # to the end of the line; a backslash at the end of a line continues it. `'x` is short for `(quote x)`.

2.2 Values

Type	Literal or constructor	`(type x)`
number, vector	`42`, `3.14`, `(vec 1 2 3)`, `(range 8)`	`scalar`, `vec`
string	`"text\n"`	`string`
symbol	`'name`, `(sym "name")`	`symbol`
list	`(list 1 "two"(vec 3 4))`	`list`
function	`(function (x)(* x x))`	`function`
nil	`nil`	`nil`

All numbers are IEEE double vectors; there is no integer type, and `floor`, `round` and `mod` do integer work. A scalar is a vector of size one, so every arithmetic and math builtin is elementwise, with size-one broadcasting:

```
var v (vec 1 2 3)
(+ v 10)                  # (11 12 13)
(* v v)                   # (1 4 9)
(sin (/ (range 8) 8))     # elementwise
(+ (vec 1 2) (vec 1 2 3))    # error: +: size mismatch (2 vs 3)
```

Lists hold anything, including other lists and vectors; that is also how matrices (a list of row vectors) and spectra (`(list re im)`) are represented. Strings are immutable; `getidx` on a string gives a one-character string.

Truth: everything is true except `0`, `nil`, an empty string, an empty list, and a vector with a zero in it. Comparisons return 1 or 0, elementwise on vectors; `==` and `equal?` compare structure on

non-numbers, and `<` orders two strings lexicographically. Constants: `nil`, `true`, `false`, `pi`, `inf`, `version`.

2.3 Variables: `var` and `set`

`var` defines a variable in the *current* environment: at top level a global, inside a function a local of that call. It never touches a variable of the same name further out, so a library's locals cannot clobber yours. `set` assigns to the nearest existing variable, however far out, and is an error if there is none.

```
var counter 0
function bump () (set counter (+ counter 1))
function shadow () {
    var counter 99          # a different, local counter
    return counter
}
bump
print counter (shadow) counter    # 1 99 1
```

Blocks (`{ }`, loop bodies, `if` branches) do not open a scope: a `var` in a `while` body inside a function is the function's local.

2.4 Control flow

```
if (< x 0) { print "negative" } { print "not negative" }    # then-block, optional else-block
while (< i 10) { var i (+ i 1) }
for (var k 0) (< k 10) (var k (+ k 1)) { print k }           # init, condition, step, body
if (== k 7) { break }         # break and continue are calls, so inside a block or in parens
```

There is no `else if`; a chain of `if`s with early `return`s reads the same way inside a function. `do` evaluates forms in order and returns the last; it is what a block is.

2.5 Functions

```
function square (n) (* n n)           # named, one-expression body
function fact (n) {                   # block body; return leaves early
    if (< n 2) { return 1 }
    return (* n (fact (- n 1)))
}
var inc (function (n) (+ n 1))        # anonymous function in a variable
(map (vec 1 2 3) square)              # functions are values
```

Closures capture the environment they were created in; a stateful closure updates a captured variable with `set`:

```
function make-counter () {
    var c 0
    return (function () { set c (+ c 1) })
}
var ctr (make-counter)
ctr                                   # 1
print (ctr)                           # 2
```

Currying. A function called with fewer arguments than it takes returns a partial application; with more, the result is applied to the rest:

```
function add3 (a b c) (+ a b c)
var add1 (add3 1)
(add1 2 3)                              # 6
(map (vec 1 2 3) (add3 10 100))         # (111 112 113)
function twice (f) (function (x) (f (f x)))
(twice inc 5)                           # 7: (twice inc) applied to 5
```

Warning: Because of currying, calling a library function with one argument too few returns a function instead of failing: `(mat-str M)` is a partial application, not a string. Functions have no optional arguments for this reason; where a default makes sense there are two functions (`dist` and `dist-p`, `mat-print` and `mat-print-with`). Builtins do check their arity.

Tail calls run in constant space: a recursive function whose recursive call is the last thing it does can loop a million times. An early `return` inside a block is rewritten to a tail call at definition time, so the natural style costs nothing; only a `return` from inside a loop unwinds.

2.6 Infix arithmetic: expr

```
(expr (x + y * 2))                      # precedence as usual: 16 for x
    =10, y=3
(expr ((x + y) * 2))                    # parentheses group
(expr ((sqrt x) + 1))                   # a form without an operator in
    second position is a call
(expr (a > 1 && b < 2))                 # comparisons and logic: < > <= >
    = == != && ||
```

2.7 Errors

```
var r (try (num "abc") catch e (concat "caught: " e))
error "negative not allowed: " n        # raise; the arguments become the
    message
assert (== (+ 2 2) 4) "arithmetic"      # std: stop unless true
```

Every error carries the file, the line and the call stack, innermost first:

```
reference.mu:426: from inner
  1> inner() at reference.mu:428
  2> outer() at reference.mu:431
```

Builtins check the number and types of their arguments (`sqrt: expected 1 argument, got 0`; `+: expected number, got string`). Deep recursion stops with `stack overflow` rather than crashing.

2.8 Code as data

```
var code '(+ 1 2)                       # a list, not a call
(eval code)                             # 3
(eval (list '* 2 (+ 1 2)))              # code built at run time: 6
(apply add3 (list 1 2 3))               # a list spread as arguments
```

2.9 Loading files

`(load "file.mu")` runs a file once in the global environment. A relative name is searched next to the loading file, in the current directory, in each entry of `MUSIL_PATH`, then in `~/.musil`. Reading functions (`read`, `read-wav`, `read-csv`, `exists?`, `stat`) apply a similar rule to their data: a relative path that does not exist in the current directory is looked for next to the file being run, so a script finds its data from wherever it was started, on the command line or from the IDE. A file that ran is not run again (so mutual loads are safe); a file that failed is not cached. Nothing is loaded automatically: a program that wants the standard library says `load "std.mu"`; each library loads what it needs (`signals.mu` loads `scientific.mu`, which loads `std.mu`).

2.10 Sharing and copying

Lists and vectors are shared by reference: `(var alias orig)` makes two names for one object, and `push` or `setidx` through either is visible through both. `(copy x)` makes a new list or vector with the same elements (a shallow copy). Strings and numbers behave as values.

3 The IDE

Musil the application is the interpreter with an editor around it.

- **Editor** (top left): `.mu` files with syntax colouring (keywords, the documented builtins, strings, numbers, comments). **Cmd-Enter** runs the block around the cursor (the lines between blank lines) or the selection; **Cmd-Shift-Enter** the whole file; **Cmd-Alt-Enter** the current line. The lines sent flash. A toolbar has Run file, Run block, Run line, Stop, Clear, Controls and Settings. Undo, find (Cmd-F, Cmd-G), comment/uncomment (Cmd-/), text size (Cmd-+, Cmd–, for every panel), files by drag and drop; Settings (Cmd-,) chooses the font, the size, line numbers, the current-line highlight, the margin guide and the evaluation port (0 turns it off), and remembers them.
- **Console** (bottom left): everything printed, and a line to type Musil into: Enter runs it, Up/Down recall history, Tab completes a name. Esc or Cmd-. stops a running program (the interpreter checks at safe points, so the session survives).
- **Variables** (top right): the globals with a preview of their values, coloured by kind (numbers, strings, lists, functions); double-click prints one.
- **Help** (bottom right): type a name or a word to search the documentation of every function. `(manual)` or the Help menu opens this PDF.
- **Plots and controls** open in windows of their own, from the IDE exactly as from the command line; several plots can be open at once.
- **From another editor**: the IDE listens on port 7770; code sent from VS Code (the extension in `editors/vscode/musil`) or from `nc` is evaluated in the same session.

The interpreter runs on its own thread; the window stays responsive while a program runs. Libraries are found inside the app bundle, so nothing needs to be installed. On macOS, `./deploy_macos.sh` builds `dist/Musil.app`.

4 Libraries

A library is a pair of files: `name.h` holds the functions that need C++, an element-by-element loop or the operating system; `name.mu` holds everything that is a composition of vector operations, which runs at the same speed in Musil. The C++ halves are registered when the

interpreter starts; the Musil half is loaded with `load "name.mu"`. Each library has a test (`tests/test_name.mu`) and a runnable reference (`examples/reference_name.mu`).

Library	Load	Contents
core	built in	the language, arithmetic, lists, vectors, strings, errors, load
std	`load "std.mu"`	vector constructors and statistics, sequences, strings, files, map/filter/red
system	`load "system.mu"`	processes, directories, CSV, WAV, UDP/OSC
scientific	`load "scientific.mu"`	matrices, decompositions, regression, PCA, k-means, KNN
signals	`load "signals.mu"`	generators, FFT, STFT, phase vocoder, features, filters, reverb
live	`load "live.mu"`	the audio device, a sample-accurate clock, voices playing buffers and files
plot	`load "plot.mu"`	figures of lines, points, bars, images and surfaces; windows and PNGs

The function references that follow are generated from the doc comment above each function in the source, the same text that `help` prints.

4.1 core

Always available, no `load` needed. Special forms are listed first.

`(quote x)`

'x: x itself, unevaluated: code as data

`(do forms...)`

evaluate in order, return the last; what { } builds

`(if cond then [else])`

cond is true unless it is 0, nil, empty, or a vector with a zero

`(while cond body)`

loop; (break) and (continue) inside it

`(for init cond step body)`

C-style loop: (for (var i 0) (< i 10) (var i (+ i 1)) { ... })

`(var name value)`

define name in the current environment; a function's var is always local

`(set name value)`

assign to the nearest existing name, however far out; error if none

`(function name (params)`

body) define a function; (function (params) body) an anonymous one

`(return [value])`

leave the function; free in tail position

`(break) (continue)`

leave or restart the innermost loop

`(try body catch e handler)`

run body; on an error bind its message to e and run handler

`(expr (a op b ...)`

) infix arithmetic: + - * / % < > <= >= == != && ||; ( ) groups, (f x) calls

`(eval form)`

evaluate a form (code built as data) in the global environment

`(apply f list)`

call f with the elements of list as arguments

`(+ a b ...)  (- a [b ...])  (* a b ...)  (/ a [b ...])`

arithmetic, elementwise with size-1 broadcasting; (- x) negates, (/ x) is the reciprocal, (+) is 0 and (*) is 1

`(< a b) (> a b) (<= a b) (>= a b)`

comparisons: elementwise 1/0 on numbers, lexicographic on two strings

`(== a b) (!= a b)`

equality: elementwise on numbers, structural on anything else

`(equal?  a b)`

structural equality as a single 1/0, also for vectors and nested lists

`(sin x) (cos x) (tan x) (asin x) (acos x) (atan x) (sqrt x) (exp x) (log x) (log2 x) (log10 x)`

trigonometry and logs, elementwise

`(abs x) (floor x) (ceil x) (round x)`

rounding and magnitude, elementwise

`(mod a b) (pow a b) (atan2 y x)`

remainder (sign follows fmod), power, two-argument arctangent; elementwise

`(not x) (and a b ...)  (or a b ...)`

logic on truth values; and/or return 1 or 0 and evaluate every argument

`(min v) (max v)`

smallest or largest element of one vector; with several arguments, elementwise over them

`(list x ...)`

a list of anything

`(cons x L)`

a new list with x in front; (append L x ...) a new list with x ... at the end

`(push L x)`

append in place, returning the list; (pop L) remove and return the last element

`(append L x ...)`

a new list with x ... added at the end

`(pop L)`

remove and return the last element of a list, in place

`(length x) (head x) (tail x) (empty? x)`

on a list, a vector or a string

`(getidx x k)`

element k of a list, vector or string; negative k counts from the end

`(setidx x k v)`

set element k of a list or vector in place

`(vec x ...)`

a vector from numbers, vectors and lists of numbers, concatenated; a scalar is a vector of size 1

`(sum v)`

sum of the elements (0 for an empty vector)

`(print x ...)`

print the arguments separated by spaces, then a newline

`(exit [code])`

end the program

`(type x)`

"scalar" "vec" "string" "symbol" "list" "function" "nil" or "opaque:tag"

`(str x)`

x printed as a string

`(sym s)`

a symbol from a string; (num s) a number from a string

`(copy x)`

a shallow copy: a new list or vector with the same elements

`(error x ...)`

raise an error with the arguments as message

`(clock)`

seconds from a monotonic clock, for timing

`(load "file.mu")`

run a file once, searched next to the loading file, in the current directory, in MUSIL_PATH and in ~/.musil; (find-file "name") the path load would use, or nil

`(defined? 'name)`

is name bound?

`(vars)`

the global names, sorted

`(help name)`

print the documentation of a builtin or library function (from help.txt)

`(num s)`

the number written in a string (or a number unchanged)

`(find-file "name")`

the path load would use for name, searched the same way, or nil

4.2 std

The general-purpose standard library. Vectors: `range` is the constructor everything else is built on (`zeros`, `ones`, `linspace`), then statistics (`mean`, `stdev`, `median`, `quantile`, `histogram`), and `interp1` for table lookup. Sequences: `slice`, `take`, `drop`, `find`, `zip`, `unique`. Strings: `split`, `join`, `format`, `fmt-fixed`. Files: `read`, `write`. Higher-order: `map`, `filter`, `reduce`, `each`, all of them on lists and vectors alike.

```
load "std.mu"
var v (range 0 10 0.5)
print (mean v) (stdev v) (quantile v 0.9)
print (map (list "a" "b") upper) (filter v (function (x) (> x 7)))
print (format "{} of {}" (count v even?) (length v))
```

`(range n)` `(range a b)` `(range a b step)`

a vector of evenly spaced numbers, end excluded

`(seed n)`

seed the random generator so that rand repeats

`(rand)`

a number in [0, 1); (rand n) a vector of n of them

`(sort v)`

the elements of a vector in ascending order

`(reverse x)`

a list, vector or string reversed

`(slice x start [count])`

part of a list, vector or string; negative start counts from the end

`(find x v)`

index of the first element equal to v, or of the substring v in a string; -1 if absent

`(concat x ...)`

the arguments printed and joined into one string

`(split s sep)`

the pieces of s between occurrences of sep; (split s "") the characters

`(join L sep)`

the elements of L printed and joined with sep

`(format "text {} more {}" x y)`

each {} replaced by the next argument

`(upper s) (lower s) (trim s)`

case changes and whitespace trimming

`(chr n)`

the character with code n

`(ord s)`

the code of the first character of s

`(read path)`

the whole file as a string; a relative path that does not exist in the current directory is looked for next to the file being run (so a script finds its data from anywhere)

`(write path x)`

write x (printed) to a file, replacing it

`(append-file path x)`

write x (printed) at the end of a file

`(exists? path)`

does the file or directory exist?

`(input [prompt])`

a line read from the console, or nil at end of input

`(map x f)`

f applied to every element of a list (any results) or vector (numeric results)

`(filter x f)`

the elements for which f is true, as a list or vector like x

`(reduce x f init)`

fold: (f (f (f init x0) x1) x2) ...

`(assert cond [message ...])`

stop with an error unless cond is true

`(sign x)`

-1, 0 or 1

`(even? n) (odd? n)`

`(between? x lo hi)`

lo <= x <= hi

`(hypot a b)`

sqrt(a^2 + b^2)

`(gcd a b) (lcm a b)`

on non-negative integers

`(lerp a b t)`

linear interpolation, t in [0, 1]

`(map-range x in-lo in-hi out-lo out-hi)`

re-map x from one range into another

`(deg->rad d) (rad->deg r)`

`(zeros n)`

vector of n zeros

`(ones n)`

vector of n ones

`(linspace a b n)`

n points from a to b inclusive

`(mean v)`

arithmetic mean

`(prod v)`

product of the elements

`(dot a b)`

inner product (sizes must match)

`(norm v)`

euclidean norm

`(cumsum v)`

running sum

`(stdev v)`

population standard deviation

`(normalize v)`

scale to [0, 1]; a constant vector becomes zeros

`(diff v)`

first differences, one element shorter

`(argmin v)` `(argmax v)`

index of the smallest / largest element

`(median v)`

the middle value (mean of the two middle values for even length)

`(quantile v q)`

value below which a fraction q of the data lies (linear interpolation)

`(histogram v bins)`

counts per bin over [min v, max v]; returns (list counts edges)

`(interp1 xs ys xq)`

linear interpolation of the table (xs, ys) at the points xq; xs must be increasing; xq outside the table is clamped to the ends

`(clip v lo hi)`

clamp every element to [lo, hi]

`(fixed x d)`

round x to d decimal places

`(last x)`

last element

`(take x n)`

first n elements

`(drop x n)`

all but the first n elements

`(vec->list v)`

the elements of a vector as a list

`(contains? x v)`

is v an element of x (or a substring, for strings)?

`(each x f)`

call f on every element, for side effects; returns nil

`(any? x f)`

does f hold for some element?

`(all? x f)`

does f hold for every element?

`(count x f)`

number of elements for which f holds

`(concat-list a b)`

a new list with the elements of a then those of b

`(flatten L)`

one level of nested lists removed

`(zip a b)`

list of two-element lists

`(unique L)`

elements in first-seen order, duplicates removed

`(sort-list L)`

sort a list of numbers or strings (insertion sort; fine for short lists)

`(min-of L)` `(max-of L)`

smallest / largest element of a list of numbers

`(repeat s n)`

string s repeated n times

`(starts-with? s prefix)` `(ends-with? s suffix)`

`(replace s old new)`

every occurrence of old replaced by new

`(count-substr s pat)`

number of non-overlapping occurrences of pat

`(pad-left s w ch)` `(pad-right s w ch)`

pad to width w with character ch

`(fmt-fixed x d)`

x as a string with exactly d decimals: (fmt-fixed pi 4) => "3.1416"

`(fmt-pct x d)`

x as a percentage string: (fmt-pct 0.752 1) => "75.2%"

`(identity x)`

`(compose f g)`

the function x -> (f (g x))

`(times n f)`

call f n times with 0 .. n-1

`(assert-near a b eps msg)`

numeric comparison within eps

`(assert-equal actual expected msg)`

structural comparison with a helpful message

4.3 system

Access to the operating system. `read-wav` returns `(list sample-rate (list channel ...))`, each channel a vector, which is the representation every signal function uses; `write-wav` takes a vector (mono) or a list of vectors. CSV tables are lists of rows, numeric cells become numbers.

```
load "system.mu"
var w (read-wav "sound.wav")
var sr (head w)
var left (head (getidx w 1))
write-wav "/tmp/out.wav" sr (* 0.5 left)
print (file-lines "notes.txt") (ls ".") (getenv "HOME")
```

`(exec "command")`

=> the command's standard output as a string

`(getenv "NAME")`

=> value or nil

`(sleep seconds)`

`(now)`

=> seconds since the Unix epoch (wall clock; clock is the monotonic one)

`(cwd)`

=> current directory

`(ls [dir])`

=> list of entry names, sorted

`(mkdir path)`

=> creates the directory and its parents; nil

`(remove path)`

=> deletes a file or an empty directory; 1 if something was removed

`(stat path)`

=> (list size is-dir modified-seconds) or nil if it does not exist

`(read-csv path)`

=> table. write-csv is in system.mu.

`(read-wav path)`

=> (list sample-rate (list channel-vector ...)); 16-bit PCM or 32-bit float WAV

`(write-wav path sample-rate channels [bits])`

=> nil. channels: a vector (mono) or a list of vectors; bits 16 (default) or 32 (float)

`(udp-send host port payload [osc?])`

=> 1 on success, 0 on failure. With osc?=1 the payload is

`(udp-receive host port [timeout-seconds])`

=> the next datagram as a string, or nil on timeout/failure

`(file-lines path)`

list of lines, without the trailing newline

`(write-lines path lines)`

write a list of lines, one per line

`(file? path) (dir? path)`

existence tests by kind

`(file-size path)`

size in bytes, or nil

`(basename path) (dirname path) (extension path)`

string surgery on paths

`(csv-escape s)`

quote a field if it holds a comma, a quote or a newline

`(write-csv path table)`

table is a list of rows, a row a list of cells

`(elapsed thunk)`

seconds taken by calling thunk

`(wav-duration w)`

seconds of a value returned by read-wav

4.4 scientific

Linear algebra, statistics and machine learning. A matrix is a list of row vectors: `(list (vec 1 2)(vec 3 4))` is 2×2; `(getidx M i)` is a row. Products, transposition, elimination (`det`, `inv`, `solve`), the symmetric eigensolver, k-means and KNN are C++; construction, elementwise arithmetic, statistics along an axis, covariance, PCA, regression and the model helpers are Musil.

```
load "scientific.mu"
var A (list (vec 2 1) (vec 1 3))
print (mat-mul A (inv A)) (solve A (vec 3 5)) (eig-sym A)
var P (pca X)                          # rows: direction then
    eigenvalue, largest first
var km (kmeans X 3)                    # (list labels centroids)
var model (knn-model training 3)
print (accuracy (knn-test model test) test)
```

`(mat-mul A B ...)`

matrix product, left to right

`(transpose M)`

`(det M)`

`(rank M)`

`(inv M)`

by Gauss-Jordan on [M | I]

`(solve A b)`

=> x with A x = b

`(median-filter v order)`

the running median of width order

`(eig-sym M)`

=> (list eigenvalues eigenvectors) of a symmetric matrix by Jacobi rotations;

`(kmeans M k)`

=> (list labels centroids): labels is a vector of cluster indices, centroids a k x d matrix

`(knn training k queries)`

=> list of predicted labels.

`(list->mat L)`

a matrix from a list of lists of numbers

`(mat-fill r c v)`

r x c matrix filled with v

`(mat-zeros r c) (mat-ones r c)`

matrices of zeros or ones

`(mat-ones r c)`

a matrix of ones

`(mat-rand r c)`

r x c matrix of values in [-1, 1]

`(eye n)`

identity; (diag v) diagonal matrix from a vector

`(diag v)`

a square matrix with v on the diagonal

`(mat-diag M)`

the diagonal of a matrix as a vector

`(nrows M) (ncols M)`

the number of rows or columns

`(ncols M)`

the number of columns

`(mat-shape M)`

(list rows cols)

`(mat-get M i j)`

element; (mat-set M i j v) in place

`(mat-set M i j v)`

set an element in place

`(mat-row M i)`

row i as a vector; (mat-col M j) column j as a vector

`(mat-col M j)`

column j as a vector

`(get-rows M from n)`

n rows starting at from, as a matrix

`(get-cols M from n)`

n columns starting at from, as a matrix

`(mat-vec A x)`

A x as a vector; (vec-mat x A) x A as a vector

`(vec-mat x A)`

x A as a vector

`(outer u v)`

the matrix u v', rows u[i] * v

`(trace M)`

sum of the diagonal

`(mat-add A B) (mat-sub A B) (hadamard A B)`

elementwise sum, difference and product of two matrices

`(mat-sub A B)`

elementwise difference

`(hadamard A B)`

elementwise product

`(mat-scale M s) (mat-shift M s)`

every element times s, or plus s

`(mat-shift M s)`

every element plus s

`(mat-map M f)`

f applied to every element

`(mat-round M d)`

every element rounded to d decimals (for printing)

`(vstack A B)`

rows of A then rows of B

`(hstack A B)`

each row of A followed by the row of B

`(col-stack vectors)`

a matrix whose columns are the given vectors: (col-stack (list x y))

`(lp-norm v p)`

(sum |v|^p)^(1/p); p = 2 is the euclidean norm

`(dist x y)`

euclidean distance; (dist-p x y p) with another p

`(dist-p x y p)`

distance with another p: 1 for the city-block distance

`(dist-matrix M)`

pairwise euclidean distances between the rows of M

`(nearest M x)`

index of the row of M closest to x

`(mat-sum M axis)`

`(mat-mean M axis)`

`(mat-std M axis)`

population standard deviation

`(zscore M)`

columns standardised to mean 0, std 1 (constant columns become 0)

`(standardize M)`

the same as zscore

`(center M)`

each column minus its mean

`(cov M)`

sample covariance of the columns (rows are observations)

`(corr M)`

correlation of the columns; constant columns give 0

`(linefit x y)`

(slope intercept) of the least-squares line through the points

`(mat-str M d)`

one line per row, d decimals, columns aligned

`(mat-print M)`

print with 3 decimals; (mat-print-with M d) chooses the decimals

`(mat-print-with M d)`

print with d decimals

`(linreg-fit X y)`

coefficients b with y ~ X b, by the normal equations; y is a vector

`(linreg-predict X b)`

X b as a vector

`(linreg-residuals X y b)`

`(add-intercept X)`

a column of ones in front of X, for a model with a constant term

`(pca X)`

=> d x (d+1) matrix: row k is the k-th principal direction followed by its eigenvalue, largest first

`(pca-directions P)`

the directions of a pca result as a d x d matrix

`(pca-eigenvalues P)`

its eigenvalues as a vector

`(pca-scores X k)`

X projected on the first k principal directions, n x k

`(pca-explained P)`

fraction of variance per component

`(kmeans-labels result)` `(kmeans-centroids result)`

the two parts of a kmeans result

`(kmeans-centroids result)`

the centroids of a kmeans result, k x d

`(cluster-sizes labels k)`

how many points fell in each of the k clusters

`(knn-model training k)`

a model: (list training k); training is a list of (list features label)

`(knn-predict model queries)`

labels for a list of feature vectors

`(knn-predict-one model q)`

`(knn-test model samples)`

predictions for samples given as (list features label)

`(accuracy predictions samples)`

fraction of predictions equal to the samples' labels

`(train-test-split samples fraction)`

(list training test), first part of the list

`(shuffle L)`

a new list in random order (Fisher-Yates)

4.5 signals

Offline signal processing on vectors. A signal is a vector of samples; a complex spectrum is `(list re im)`; a polar spectrum is `(list mag phase)`. The FFT, the table oscillator, the IIR filter, delay, resampling, autocorrelation, peak picking and gather are C++; generators, windows, convolution, the STFT and its inverse, cepstral envelopes, the phase vocoder, the spectral and temporal descriptors, the RBJ biquads, comb and allpass sections and the Schroeder reverb are Musil, as compositions of vector operations.

```
load "signals.mu"
var x (sine 44100 440 1)
var frames (stft x 2048 512)
var y (istft frames 2048 512)                # exact reconstruction
var slow (pvoc-stretch x 2)                   # twice as long, same
    pitch
var up (pvoc-pitch-formant x 1.5 400)         # a fifth up, formants
    kept
var any (pvoc x (list (list "stretch" (list 1 3)) (list "pitch" 0.75)
    (list "envelope" 400)))
```

```
var lp (lowpass x 44100 500 0.707)
print (spectral-centroid (magnitude-spectrum x) (spectrum-freqs 65536
    44100))
```

The phase vocoder is one engine, `pvoc`, in C++ (a frame is sixty vector operations on 4096 bins, ten thousand times a minute): a fixed synthesis hop with the analysis hop derived from the stretch, zero-phase windowing, phases locked to the nearest spectral peak, pitch shift and formant move in the spectrum, envelope preservation through the cepstral envelope, denoising, three cross-synthesis modes and two phase effects, every parameter able to ramp over the file. `pvoc-stretch`, `pvoc-pitch`, `pvoc-pitch-formant`, `pvoc-formants`, `pvoc-cross`, `denoise`, `robotize` and `whisperize` are one-line wrappers around it.

`(fft signal)`

=> (list re im), each of length next-pow2(length); the signal is zero-padded

`(ifft (list re im)`

) => real signal of the same length (a power of two)

`(osc sr freqs table)`

=> one sample per element of freqs, reading the wavetable with linear

`(iir x b a)`

=> y with a0 y[n] = sum b[k] x[n-k] - sum a[k>0] y[n-k]; a[0] need not be 1

`(delay x d)`

=> x delayed by d samples (fractional, linear interpolation), same length

`(resample x factor)`

=> x with its length multiplied by factor, by windowed-sinc interpolation

`(autocorr x)`

=> biased autocorrelation (sum / n) for lags 0 .. n/2 - 1; it decays with lag, which

`(interleave (list a b ...)`

) => one vector a0 b0 a1 b1 ...

`(deinterleave v channels)`

=> (list a b ...): the channels of an interleaved vector

`(local-maxima v)`

=> indices k with v[k] > v[k-1] and v[k] > v[k+1] (interior points only)

`(gather v indices)`

=> the elements of v at the given (integer) indices, as a vector

`(add-at!  dst pos src)`

=> dst, with src added in place starting at pos; dst must be long enough.

`(comb x d g)`

=> feedback comb y[n] = x[n] + g y[n-d]; (allpass x d g) => Schroeder allpass

`(pvoc x opts)`

after sparkle: fixed synthesis hop I = window/overlap, analysis hop D = I/stretch

`(adsr sr gate a d s r)`

=> an envelope following a gate signal (1 = on, 0 = off): attack a and

`(lag x sr seconds)`

=> x smoothed by a one-pole lowpass with the given time constant (parameter

`(sine sr freq dur)`

a sine of freq Hz lasting dur seconds

`(noise n)`

white noise in [-1, 1]

`(impulse n)`

a 1 followed by n-1 zeros

`(gen n amps)`

one period of a wavetable with the given harmonic amplitudes, plus a guard point (n+1 samples, last = first), normalised to a peak of 1; for osc

`(oscbank sr amps freqs table)`

sum of oscillators; amps and freqs are lists of per-sample envelopes

`(mix layers)`

overlay (list (list position signal) ...) into one signal

`(add-at dst pos src)`

dst with src added starting at pos; dst grows if needed

`(pan x pos)`

a mono signal to (list left right), equal power; pos from -1 (left) to 1 (right)

`(fade-in x n) (fade-out x n)`

linear fades over n samples

`(rms x)`

root mean square

`(normalize-peak x)`

scale so the peak is 1; (normalize-rms x target) so the rms is target

`(normalize-rms x target)`

scale so the rms is target

`(db x) (undb d)`

amplitude to decibels and back

`(next-pow2 n)`

the smallest power of two >= n (fft sizes)

`(window n a0 a1 a2)`

generalised cosine window, periodic (so hann overlap-adds exactly at hop n/4); (hann n) (hamming n) (blackman n) are the usual ones

`(hann n) (hamming n) (blackman n)`

the usual windows, periodic

`(hamming n)`

the Hamming window

`(blackman n)`

the Blackman window

`(car->pol spec)`

(list re im) -> (list mag phase); (pol->car) the other way

`(pol->car spec)`

(list mag phase) -> (list re im)

`(magnitudes spec) (phases spec)`

from a complex spectrum

`(spectrum-freqs n sr)`

the frequency of each of the first n/2 bins

`(magnitude-spectrum x)`

the positive-frequency magnitudes of x, normalised so a full-scale sine reads 1

`(complex-mul a b)`

product of two (list re im) spectra

`(conv x h)`

linear convolution through the FFT, length (+ (length x) (length h) -1)

`(cepstrum mags)`

real cepstrum of a full (symmetric) magnitude spectrum

`(spectral-envelope mags order)`

smooth envelope of a full magnitude spectrum: the "true envelope", a cepstral smoothing (first ‘order‘ coefficients) iterated so that it rides on the harmonic peaks instead of averaging peaks and valleys. The order sets the resolution: about sr / (2 f0) of the sound (e.g. 150 for a voice at 44.1 kHz) follows the formants without rippling at the harmonics

`(flatten-spectrum mags order)`

the magnitudes divided by their envelope: the fine structure alone

`(impose-envelope mags source order)`

mags reshaped to carry the spectral envelope of source

`(stft x n hop)`

=> list of (list re im); (istft frames n hop) => signal by overlap-add

`(istft frames n hop)`

the signal back from stft frames by windowed overlap-add

`(stft-magnitudes frames)`

list of positive-frequency magnitude vectors, one per frame

`(princarg x)`

wrap a phase to (-pi, pi]

`(opt opts key default)`

the value of key in a list of (list key value) options, or default

`(ramp v)`

a parameter given as a number or as (list start end) -> (list start end)

`(fftshift v)`

rotate a vector by half its length (zero-phase windowing)

`(pvoc-stretch x factor)`

time-stretch by factor with the pitch kept

`(pvoc-pitch x ratio)`

pitch-shift by ratio at the same length (formants move too)

`(pvoc-pitch-formant x ratio order)`

pitch-shift keeping the formants (order about sr/100: 400 at 44.1 kHz)

`(pvoc-formants x ratio order)`

move the formants by ratio, the pitch kept

`(pvoc-cross x y mode amount order)`

cross synthesis of x by y: mode 1 multiplicative, 2 spectral flattener (y's envelope on x), 3 morphing

`(robotize x)`

zero phases: a buzz at the frame rate

`(whisperize x)`

random phases: the spectral envelope on noise

`(denoise x threshold)`

magnitudes below threshold x the peak are zeroed (0.01 - 0.1)

`(gate-spectrum mags threshold)`

magnitudes below threshold times the frame's peak set to zero

`(spectral-morph a b t n hop)`

between two sounds of the same length: magnitudes interpolated linearly, phases taken from a below t = 0.5 and from b above

`(spectral-moment amps freqs order centroid)`

weighted moment of the frequencies about a centroid

`(spectral-centroid amps freqs)`

amplitude-weighted mean frequency

`(spectral-spread amps freqs)`

standard deviation around the centroid

`(spectral-skewness amps freqs)`

asymmetry of the spectrum about its centroid

`(spectral-kurtosis amps freqs)`

peakedness of the spectrum about its centroid

`(spectral-flux amps previous)`

rectified increase from the previous frame

`(spectral-irregularity amps)`

sum of absolute differences between neighbouring bins

`(spectral-decrease amps)`

weighted decrease relative to the first bin

`(spectral-flatness amps)`

geometric over arithmetic mean: 1 for noise, 0 for a pure tone

`(spectral-rolloff amps freqs fraction)`

frequency below which the fraction of the energy lies

`(hfc amps)`

high-frequency content

`(zcr x)`

zero-crossing rate per sample

`(acf-f0 x sr)`

fundamental by autocorrelation; 0 when no clear peak

`(biquad type sr f0 q gain-db)`

=> (list b a) of an RBJ biquad; type is one of "lowpass" "highpass" "bandpass" "notch" "peak" "lowshelf" "highshelf"

`(apply-filter x coeffs)`

run x through (list b a) as returned by biquad

`(lowpass x sr f0 q) (highpass x sr f0 q) (bandpass x sr f0 q) (notch x sr f0 q)`

one biquad, applied

`(highpass x sr f0 q)`

a highpass biquad, applied

`(bandpass x sr f0 q)`

a bandpass biquad, applied

`(notch x sr f0 q)`

a notch biquad, applied

`(peak-eq x sr f0 q gain-db) (lowshelf x sr f0 q gain-db) (highshelf x sr f0 q gain-db)`

the equaliser biquads, applied

`(lowshelf x sr f0 q gain-db)`

a low shelf, applied

`(highshelf x sr f0 q gain-db)`

a high shelf, applied

`(dc-block x)`

remove the DC offset; (dc-block-r x r) with pole radius r (default 0.995)

`(dc-block-r x r)`

a DC blocker with pole radius r

`(reson x sr freq tau)`

two-pole resonator at freq Hz with decay time tau seconds; the output lasts tau seconds, so a short excitation rings out

`(schroeder-reverb x sr rt60)`

four parallel combs into two allpasses; rt60 is the decay time in seconds (how long the tail takes to fall by 60 dB). The output is the input plus rt60 seconds, so the tail is not cut off.

`(resample-to x sr-in sr-out)`

`(bpf start segments)`

break-point function: piecewise linear segments as one vector; segments is a list of (list length end): (bpf 0 (list (list 4 1) (list 4 0))) rises then falls. Each segment's end value is excluded (it starts the next one).

`(envelope-follow x n)`

rms over hops of n samples, one value per hop

`(envelope-from-values v hop)`

piecewise-linear signal through successive values, hop samples apart

4.6 live

Real-time sound. The interpreter never runs on the audio thread: it posts commands (play this buffer at this time, stop that voice, set the gain) into a lock-free queue, and the audio callback drains the queue, advances a sample clock and mixes the voices into an N-channel bus. A voice plays a buffer (mono, or a list of channels) at its own sample rate, with amplitude, pan, rate and loop, from a start time on the clock, so sounds asked for slightly ahead land exactly where asked however late the program is. The device is miniaudio, with no window needed; a silent "null" device (the option `(list (list "device""null"))`, or the environment variable `MUSIL_NULL_AUDIO`) runs the same callback from a timer, which is how the test suite exercises everything but the ears.

```
load "live.mu"
audio-init                                      # 44100 Hz, 256 samples,
    stereo, started
var v (play-file "data/drums.wav")              # a voice
wait-for v
var lp (loop-buffer x sr)
set-rate lp 0.5
stop lp
var t (audio-time)
each (range 8) (function (k) (play-at (+ t (* k 0.25)) hit sr))    #
    sample-accurate, ahead of time
audio-quit
```

Synths. An instrument is an ordinary Musil function of its parameters whose body combines the *streamable* builtins: `osc`, `noise`, `adsr`, `lag`, the biquads (`lowpass` ... `highshelf`), `iir`, `dc-block`, `delay`, `comb`, `allpass`, `conv` (partitioned when streaming), `pan`, `list`, arithmetic and elementwise math (the list is `(streamable)`). Called normally it returns a buffer, because every one of those is a vector function in `signals`. Handed to `(synth f)` its body is compiled into a graph of nodes that run *the same loops* from `signals.h` one block at a time, and its parameters

become hot: `(set-param id 'cutoff 200 0.5)` ramps the running sound; a parameter named `gate` is what `note-on`/`note-off` set. `synth-render` runs the graph offline with parameter curves, and the test suite checks it equals the direct call to float precision. Anything not streamable (`pvoc`, `resample`, ...) is refused by name; it is still a fine offline instrument.

```
function lead (gate freq cutoff) \
    (pan (* (adsr sr gate 0.01 0.2 0.5 0.4) (lowpass (osc sr freq
       saw-table) sr cutoff 2)) 0)
var a (synth lead)                                  # compiled, silent
set-params a (list (list 'freq 110) (list 'cutoff 1200))
note a 0.5                                          # gate on, off after
   half a second (scheduled)
set-param a 'cutoff 300 1.5                         # ramp while it plays
var y (lead gate-vector freq-vector 1200)           # the same function,
   offline: a buffer
```

Loops. A loop is a function of the cycle number returning events, each `(list beat dur thunk)`; `(live-loop 'bass 4)` calls the function *named* `bass` every four beats, slightly ahead of the clock (`lookahead`), and the thunks schedule sounds sample-accurately (`synth-ev`, `play-ev`, or your own `note-at`/`play-at`). Because the function is looked up by name at each cycle, redefining it while the loop runs is the live coding: the next cycle plays the new definition. `tempo`, `beat`, `stop-loop`; and the transformations of event lists, `fast`, `slow`, `shift`, `rev`, `every`, `degrade`, `cat`, `stack`, `steps`, are plain list functions. The scheduler runs in the interpreter's idle time (while the REPL or the IDE waits, and inside `sleep`), on one thread with everything else.

```
tempo 110
function drums (cycle) (stack (list
    (steps 1 (list 1 nil 1 nil) (function (x) (function (t d) (note-at
        t k 0.1))))
    (every 4 cycle (function (e) (fast e 2)) (steps 0.5 (list nil 1) (
       function (x) (function (t d) (note-at t h 0.05)))))))
live-loop 'drums 4
function drums (cycle) ...          # redefine: next cycle it changes
```

Controls and OSC. `(control 'cutoff 100 5000 800)` declares a named value with a range, `(toggle 'on 1)` an on/off one; `bind-control` makes synth parameters follow it, `set-control` and `control-value` read and write it from code, `(controls)` opens the window of sliders and check boxes and returns. `osc-listen` and `osc-map` let an incoming OSC message move a control; `osc-send` sends one; `osc-encode` and `osc-decode` are OSC in the language, over `udp-send` and `udp-receive`, for anything the conveniences do not cover. None of this is the evaluation port below: OSC carries numbers to controls over UDP, the port carries code over TCP.

Editors. `(serve 7770)` (the IDE does it at start-up; the command line with `-serve 7770 -i`) opens a local port that evaluates the text it receives, in the session, as if typed. The VS Code extension in `editors/vscode/musil` sends the block around the cursor with Cmd-Enter (the file with Cmd-Shift-Enter) and gives `.mu` files syntax colouring, from a grammar generated with `help`. From a shell, `musil -send 7770 block.mu` (or with the code on stdin) sends and prints the reply, and any editor that can run a command can do the same; the two Musil processes stay separate sessions, the port only lets one hand code to the other.

`(audio-open sr block channels [opts])`

> open the audio device; if one is already open with the same sample rate and channels it is reused (voices cleared), otherwise it is closed and reopened. opts: (list "device" "null") for the silent backend (tests, headless machines). Fails if a device is already open.

`(audio-close)`

stop and close the device

`(audio-start) (audio-stop)`

run or pause the callback; the clock advances only while running

`(audio-status)`

=> (list (list "open" 0/1) (list "running" 0/1) (list "sr" sr) (list "block" n) (list "channels" n) (list "device" name) (list "time" seconds) (list "voices" n) (list "load" fraction) (list "peak" x))

`(audio-devices)`

=> list of the playback device names

`(audio-time)`

=> seconds on the engine's clock (sample-accurate, advances while running)

`(play-buffer buffer sr amp pan rate loop at)`

=> voice id. buffer is a vector (mono) or a list of channel vectors, at its own sample rate; at is the start time in seconds on the clock (0 = now)

`(stop voice)`

stop one voice; (stop-all) every voice

`(master-gain g)`

the output gain, applied to the whole bus

`(voice-set voice "amp"|"pan"|"rate" value)`

change a playing voice, ramped over a few blocks

`(voices)`

=> the ids of the voices playing or scheduled

`(synth f)`

=> id: compile the function f into a running instrument (silent until its parameters say otherwise). f's parameters become hot parameters; its body may use only the streamable builtins (see (streamable))

`(set-param id name value [ramp-seconds])`

change a synth's parameter, immediately or ramped

`(synth-render f params seconds [sr])`

=> the sound of f as the engine would stream it, computed offline block by block with its parameters set to params (a list of (list name value) or (list name buffer) for a value per sample); the same graph as (synth f), so the result equals f called on the same signals. No device needed.

`(free id)`

remove a synth from the graph

`(synth-params id)`

=> the parameter names of a synth, in the order of its function

`(synths)`

=> the ids of the running synths

`(streamable)`

=> the names of the builtins a synth function may use

`(tempo bpm)`

set the tempo; beats are counted from now

`(beat)`

=> the current beat (a number, fractional); (beat-time b) => the clock time of beat b; (bpm) => the tempo

`(live-loop name beats)`

start (or restart) the loop named name: the function of that name is called every cycle of 'beats' beats; (stop-loop name) (stop-loops) (loops) => the names

`(lookahead seconds)`

how far ahead of the clock loops are scheduled (default 0.25)

`(control name lo hi value)`

declare a control (a range and a value); (toggle name value) an on/off one

`(set-control name value) (control-value name) (controls-list)`

=> (list (list name lo hi value toggle?) ...)

`(bind-control name synth-id param)`

a synth parameter follows the control (several bindings allowed); (unbind-control name)

`(clear-controls)`

forget every control

`(osc-encode address args...)`

=> the bytes of an OSC message as a string, for udp-send; (osc-decode bytes) => (list address args...) or nil: OSC in the language, on top of udp-send and udp-receive; osc-send, osc-listen and osc-map are conveniences over them

`(osc-send host port address args...)`

send an OSC message (numbers as floats, anything else as strings)

`(osc-listen port)`

receive OSC on a port (polled in the background); (osc-map "/address" control-name) the first number of a message at that address sets the control; (osc-stop)

`(audio-init)`

open the default device at 44100 Hz, 256 samples, stereo, and start it

`(audio-init-with sr block channels)`

the same with your numbers. With the environment variable MUSIL_NULL_AUDIO set (the test suite does), the silent null device is used. A device left open by a stopped program is reused (or reopened if the numbers differ), so running one live example after another just works.

`(audio-quit)`

stop everything and close the device

`(audio-sr) (audio-channels) (audio-running?)`

from audio-status

`(audio-report)`

print the status, one line per field

`(play buffer sr)`

play a buffer (a vector, or (list left right ...)) recorded at sr; => voice id

`(play-with buffer sr opts)`

the same with options: (list (list "amp" 0.5) (list "pan" -1) (list "rate" 2) (list "loop" 1) (list "at" 0.5)); at is a time in seconds on the audio clock

`(play-file path)`

play a WAV file (any channels) at its own sample rate; => voice id

`(play-file-with path opts)`

`(play-at t buffer sr)`

play at time t (seconds on the audio clock): sample-accurate however late the program is, as long as t is still ahead

`(loop-buffer buffer sr)`

play looping; stop it with (stop voice)

`(set-amp voice a) (set-pan voice p) (set-rate voice r)`

change a playing voice, ramped

`(wait-for voice)`

return when the voice has finished (polls the engine)

`(wait-until t)`

return when the audio clock has passed t seconds

`(sequence steps)`

play (list (list time buffer sr) ...) at their times, all scheduled at once

`(note-on id) (note-off id)`

set the synth's gate parameter to 1 or 0

`(note id dur)`

gate on now, off after dur seconds (sample-accurate, scheduled)

`(note-at t id dur)`

the same at time t on the clock

`(set-params id pairs)`

several parameters at once: (list (list 'freq 440) (list 'cutoff 800))

`(play-synth f pairs)`

compile f, set its parameters, gate it on; => id

`(free-all)`

free every synth

`(synth-render-file f pairs seconds path)`

render a synth offline through its graph and write a WAV

`(ev beat dur thunk)`

an event

`(synth-ev beat dur id pairs)`

a note on a synth: sets the parameters and gates it for dur

`(play-ev beat buffer sr)`

a buffer at a beat; (play-ev-with beat buffer sr opts) with play-with's options

`(ev-beat e) (ev-dur e) (ev-thunk e)`

`(shift events beats)`

move every event later by beats

`(fast events factor)`

squeeze the events in time by factor (2 = twice as fast); (slow events factor)

`(rev events length)`

play the cycle backwards (length = the loop's beats)

`(every n cycle f events)`

apply f to the events on every n-th cycle (cycle is the loop's cycle number)

`(degrade events p)`

keep each event with probability p

`(cat lists length)`

several cycles' worth of events one after another, each of length beats

`(stack lists)`

several event lists at once

`(steps beats-per-step items thunk-of-item)`

one event per item, evenly spaced; nil items are rests

4.7 plot

A figure is `(list title layers options)`; `plot.mu` builds it and `show` opens it in a window of its own and returns, from the command line and from the IDE alike; the interpreter keeps the windows alive while it waits or sleeps, so several plots can be open while a program goes on, and a script that ends with windows open stays until they are closed. `save-png` writes a file. `subplots` puts several figures in one grid. Keys in a window: `+`/`-` zoom, W A S D pan, arrows orbit a surface, `0` or `r` reset, `e` exports a PNG, Esc or `q` closes; the mouse drags to pan or orbit, the wheel zooms. Layers are lines, points, bars, an image (a matrix as a heat map) or a surface (a matrix in 3D). The one-call helpers build and show in one go; the ready-made figures know about signals and data.

```
load "plot.mu"
var fig (figure "two sines")
add-line fig nil (sin (/ (range 100) 8)) "sin"
add-line fig nil (cos (/ (range 100) 8)) "cos"
set-labels fig "sample" "value"
show fig
save-png (spectrogram x sr 1024 256) "/tmp/spec.png"
show (scatter-clusters petals (kmeans-labels km) 3)
```

`(save-png fig path [w h])`

render a figure to a PNG file (900 x 560 by default)

`(show fig [w h])`

open the figure in a window of its own and return; the program goes on and the window stays (the idle hook keeps it alive); several can be open. Skipped when MUSIL_NOSHOW is set.

`(plot-windows)`

=> how many plot windows are open; (close-plots) closes them

`(figure title)`

an empty figure

`(fig-title fig) (fig-layers fig) (fig-options fig)`

the three parts of a figure

`(fig-layers fig)`

the layers

`(fig-options fig)`

the options

`(add-line fig x y label)`

x may be nil to use 0, 1, 2, ...

`(add-scatter fig x y label)`

points; (add-bars fig x y label) bars

`(add-bars fig x y label)`

bars

`(add-image fig M)`

a matrix as a heat map, row 0 at the bottom

`(add-surface fig M)`

a matrix as a 3D surface (a figure with a surface shows only that)

`(add-layer fig layer)`

add a raw layer (list kind data ...)

`(x-or-index x y)`

x, or 0 1 2 ... when x is nil

`(set-option fig key value)`

keys: "xlabel" "ylabel" "xmin" "xmax" "ymin" "ymax"

`(set-labels fig xlabel ylabel)`

axis labels; (set-xrange fig lo hi) (set-yrange fig lo hi) axis limits

`(set-xrange fig lo hi)`

x-axis limits

`(set-yrange fig lo hi)`

y-axis limits

`(subplots title figs rows cols)`

a grid of figures, drawn together; rows or cols may be 0 (auto)

`(add-subplot fig sub)`

add one more figure to a grid

`(plot y)`

or (plot-xy x y) a line

`(plot-xy x y)`

a line of y against x

`(plot-lines ys labels)`

several lines sharing the index axis

`(scatter x y) (bars y) (image M) (surface M)`

build and show a one-layer figure

`(bars y)`

build and show a bar chart

`(image M)`

build and show a heat map

`(surface M)`

build and show a 3D surface

`(waveform x sr)`

a signal against time in seconds

`(spectrum x sr)`

magnitude spectrum in dB against frequency

`(spectrogram x sr n hop)`

STFT magnitudes in dB as an image: time along x, frequency along y

`(scatter-clusters X labels k)`

the rows of an n x 2 matrix coloured by cluster

`(histogram-figure v bins)`

a bar chart of (histogram v bins)

5 Extending Musil from C++

The interpreter is one header. A host includes it, registers functions, and runs code:

```
#include "musil.h"
musil::Interp I;
musil::make_env(I);                          // the C++ halves of the
    libraries
I.def("hello", [](musil::vlist& a, musil::Interp& i) -> musil::vptr {
     return musil::v_str("hello, " + i.str(a[0]));
}, 1, 1);                                    // name, function, min and
    max arguments
I.run("print (hello \"world\")");
```

A builtin receives its evaluated arguments as a `vlist` and the interpreter. `eval` has already checked the argument count. Inside, `i.num(v)`, `i.scalar(v)`, `i.index(v)`, `i.str(v)`, `i.list(v)` and `i.fn(v)` return the payload or raise a Musil error naming the builtin; `i.bad("message")`

raises one with your text. Values are made with `v_num`, `v_arr`, `v_str`, `v_list`; `i.call_fn(f, args)` calls a Musil function; `i.yield_fn` is called every 1024 evaluations for hosts that service an event loop.

Document a builtin with a comment above it in the form `// (name args) description`; `tools/gendoc.py` turns those comments into `help.txt` and into this manual. A library is complete when it has `name.h`, `name.mu`, `tests/test_name.mu` and `examples/reference_name.mu`.

6 Examples

All in `examples/`, all run by `ctest`. Run them from that directory.

`reference*.mu`	one runnable tour per library
`hello, if, while, procs, math, strings, arrays, files`	the language, one topic each
`music_demo.mu`	temperament, scales, pitch-cla
`chords.mu`	pitch-class sets as vectors: dis
`matrices.mu`, `iris.mu`	linear algebra; PCA, k-means
`stft, filters, reverb, cross_synth, transients, additive, features`	signal processing on recording
`phase_vocoder.mu`	time stretching, pitch shifting
`plots.mu`, `iris_plot.mu`	waveforms, spectrograms, resp
`live_play.mu`, `live_chorale.mu`, `live_probe.mu`	sounds through the device; a
`live_synth.mu`, `live_offline.mu`	instruments as functions, play
`live_loops.mu`, `live_controls.mu`, `live_session.mu`	patterns redefined while they